```
};

struct node
{
    std::atomic<T*> data;
    std::atomic<node_counter> count;    <-- (3)
    counted_node_ptr next;

    node()
    {
        node_counter new_count;
        new_count.internal_count=0;
        new_count.external_counters=2;    <-- (4)
        count.store(new_count);

        next.ptr=nullptr;
        next.external_count=0;
    }
};
```

In listing 7.15, `tail` is now an `atomic<counted_node_ptr>` the same as `head` ❶, and the `node` structure has a `count` member to replace the `internal_count` from before ❸. This `count` is a structure containing the `internal_count` and an additional `external_counters` member ❷. Note that you need only 2 bits for the `external_counters` because there are at most two such counters. By using a bit field for this and specifying `internal_count` as a 30-bit value, you keep the total counter size to 32 bits. This gives you plenty of scope for large internal count values while ensuring that the whole structure fits inside a machine word on 32-bit and 64-bit machines. It's important to update these counts together as a single entity in order to avoid race conditions, as you'll see shortly. Keeping the structure within a machine word makes it more likely that the atomic operations can be lock-free on many platforms.

The `node` is initialized with the `internal_count` set to zero and the `external_counters` set to 2 ❹ because every new node starts out referenced from `tail` and from the `next` pointer of the previous node once you've actually added it to the queue. `push()` itself is similar to listing 7.14, except that before you dereference the value loaded from `tail`